

Quadruped Locomotion in Rough Terrain and Confined Environments

Kaavish Report
presented to the academic faculty
by

Hania Kashif	hk08454
Raahim Hashmi	rh08461
Syed Zuhair Abbas Rizvi	sr07889



In partial fulfillment of the requirements for
Bachelor of Science
Computer Science

Dhanani School of Science and Engineering

Habib University
Spring 2026

Quadruped Locomotion in Rough Terrain and Confined Environments

This Kaavish project was supervised by:

Dr. Syeda Saleha Raza
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

Dedication

Our work is dedicated to those that don't bend, don't water it down, don't try to make it logical, don't edit their own souls according to the fashion; rather, they follow their most intense obsessions mercilessly.

(Quote taken from Franz Kafka)

Acknowledgements

There are far too many wonderful people to thank here that made this project not just possible but victorious. Some of these, however, deserve special mention, so here they are.

We begin with the titular protagonist: the *Leegit Robot*, also known as the Lynx-motion T-Hex 3DoF Hexapod. Our gratitude is insurmountable towards it for existing in the first place to set our FYP into motion. Its resilience throughout our countless experiments, failed and successful, will be remembered dearly. We hope that whoever inherits it for future work will take it to grander heights.

Next, we would like to thank our supervisors Dr. Saleha Raza and Dr. Basit Memon for their unwavering support and guidance, amid uncountable difficulties particularly during inception, throughout Kaavish. It was because of their insights and feedback that we stayed focused as we journeyed through the trials and tribulations of our capstone. Additionally, we are also grateful to our co-supervisor Mr. Tabsheer Ali Askari for his devotion to planning and assessing progress as well as managing the testbed developments.

Beyond that, we must mention our university friends for sticking around and being fun to talk to about their own capstones, wonderful and not-so-wonderful coursework, plans after graduation, and sundries. Similarly, we extend our gratitude towards the many brilliant faculty from DSSE and AHSS both that, with their teachings within and outside the classroom from algorithms to art, inspired us to become who we are. Furthermore, we cannot forget the patience and moral support of our families during times of distress, discombobulation, and disenchantment. And, to all the lovely folk of the world who produced art, music, film, games, and literature that remind us of what it means to live life for beyond university and work, thank you.

Lastly, to the reader, who may be a student looking for inspiration on their own FYP, know that the capstone will be, quite literally, an experience to remember for the rest of your **life**.

Abstract

Mobile robots are widely used for inspection, exploration, and search and rescue missions. Legged robots, in contrast to wheeled robots, have the added benefit of legged locomotion which allows them to tackle a wide variety of different terrains that a wheeled robot cannot. Caves, abandoned tunnels, and collapsed buildings are examples of environments that consist of such terrain. Using an existing hobby-grade quadruped robot kit, we sought to make it capable of traversing rough terrain and navigate through confined spaces. We created a complete simulation package for the robot, which was previously missing. We augmented the hardware kit with mounts for our compute modules and sensors to obtain feedback required for robust locomotion. For locomotion in rough terrain, we tested classical control techniques, learning-based approaches, and evolutionary algorithms. To allow the robot to navigate through confined spaces, like crawling under an obstacle or squeezing through a narrow gap, we used a depth camera for perceiving the immediate terrain and deciding on a posture that was used to constrain the locomotion algorithms to ensure the robot moved with a height and width that did not collide with the environment. Thanks to our simulation package, we were able to rapidly test our algorithms, without the added hassle and risk of testing on hardware.

Contents

1	Introduction	10
1.1	Problem Statement	10
1.2	Proposed Solution	10
1.3	Intended User	11
1.4	Project Gantt Chart and Deliverables	11
1.4.1	Kaavish I	11
1.4.2	Kaavish II	12
1.5	Key Challenges	13
2	Literature Review	15
2.1	Robot Design	15
2.2	Locomotion and Perception	17
3	Software Requirement Specification (SRS)	21
3.1	Functional Requirements	21
3.1.1	Simulation	21
3.1.2	Perception	21
3.1.3	Locomotion	22
3.2	Non-Functional Requirements	22
3.3	Use Cases	23
3.4	Block Diagrams	24
3.4.1	Perception	24
3.4.2	Locomotion	25
3.4.3	Hardware	25
4	Software Design Specification (SDS)	28
4.1	Software Design	28
4.2	Data Design	28
4.3	Technical Details	30
5	Methodology, Experiments and Results	33
5.1	Methodology and Experiments	33
5.1.1	Kinematics	33
5.1.2	Simulation	35
5.1.3	Open-loop kinematic gait	36

5.1.4	Deep Reinforcement Learning-based Controller	37
5.1.5	MIT Cheetah 3-like Controller	38
5.1.6	Posture Adaptation	40
5.2	Results	41
6	Conclusion and Future Work	44
7	Reflection	45
	References	46

List of Figures

1.1	The Gantt chart for scheduling tasks per team member.	13
3.1	Use Case Diagram for the Legged Robot System.	23
3.2	Perception Diagram for the Legged Robot System.	24
3.3	Kinematic Locomotion Diagram for the Legged Robot System.	25
3.4	Dynamic Locomotion Diagram for the Legged Robot System.	26
3.5	DRL Locomotion Diagram for the Legged Robot System.	26
3.6	Hardware Diagram for the Legged Robot System. Lines indicate the flow of data, with dotted lines indicating the robot states or world states being observed	27
4.1	Class Diagram for the Legged Robot System.	29
4.2	Sequence Diagram for the Locomotion Pipeline, detailing the interaction between the operator, robot, and core control modules.	30
4.3	Flowchart for the Perception Pipeline, illustrating the steps from data capture to posture adaptation decision.	31
4.4	Sequence Diagram for the Hardware Pipeline, showing the flow of data and commands between physical components.	32
5.1	T-Hex (as seen in SolidWorks) with its middle legs removed, making it a quadruped.	33
5.2	Interior of cave generated using PLUME [28]	35
5.3	Visualization of Bezier Curve with $N = 64, X = 15, S = -7, A = 3, T = 6$ in the hip's Y-Z plane (left) and X-Z plane (right). The red point indicates the location of where the hip frame was placed. The green crosses (in the left plot) indicate the bezier curve control points P, Q , and R	37
5.4	A live cave pointcloud, its mapped floors and ceiling, and best z-values per safe zone.	42
5.5	Snapshot of the robot walking in the simulated cave.	43

List of Tables

5.1	Crawl Gait Schedule	36
5.2	Rewards and Penalties for Training DRL-based Controller	39

1. Introduction

1.1 Problem Statement

Mobile robots have widespread applications in industrial inspection, terrestrial and extraterrestrial exploration, and search and rescue missions. Wheeled or tracked systems are the most prevalent types of mobile robots that are being used for these tasks [37]. Such robots typically adapt to changes in terrain through passive mechanical feedback [8] and require a continuous path of support with minimal obstructions [62] which restricts the kind of environments such systems can tackle to a minority of Earth’s total landmass [61]. Several critical environments, such as caves, forests, collapsed buildings, etc, pose challenges to these systems or render them entirely unusable. At the same time, these environments’ terrain pose little challenge to animals, particularly ones that use legs for locomotion. Inspired from such animals, legged robots are an alternative category of mobile robots that are highly attractive due to their ability to traverse challenging terrain. Unlike wheeled or tracked systems that require a long patch of unobstructed terrain, legged robots only require isolated footholds to be unobstructed, greatly increasing their versatility in highly cluttered environments and rough terrain. Furthermore, they can climb over obstacles or squeeze through tight gaps by changing their morphology, similar to legged animals.

The current state-of-the-art robots such as Boston Dynamics’ Spot [72], ANY-Botics’ ANYmal [35], and Unitree’s Go Series quadrupeds are used for assisting or even replacing humans in a variety of tasks due to their impressive locomotion capabilities [74, 73, 13, 52, 29, 82, 32, 84]. Such high-end systems cost thousands of dollars [39] due to expensive actuators, sensors, and compute modules.

We define, therefore, the problem statement of this capstone project as follows:

How can we make a low-cost quadruped capable of traversing rough terrain in a purely proprioceptive manner? Furthermore, if we simulate a massless depth camera on the robot, how can we make the robot capable of navigating confined spaces using perception-based locomotion?

1.2 Proposed Solution

We use a hobbyist robot kit, a Lynxmotion T-Hex 3DoF Hexapod repurposed into a quadruped, in order to demonstrate an inexpensive yet effective alternative for such robots. Our vision is that the robot, with teleoperation, would be able to handle varied rough terrain in real life while extending its behavior to perceive close

environments using a camera and adapt its posture to minimize collisions while maintaining consistent and safe locomotion. Aside from proving that our locomotion pipeline is capable of working in even hobby-grade robots with adequate results, our proposed pipeline remains agile, versatile, and capable of further research and specialization for use cases such as search-and-rescue, cave exploration, and sundries.

1.3 Intended User

Our robotics systems’s intended primary users are: researchers on robotics, academic developers, and operators looking for teleoperable platforms capable of exploring rough, confined environments such as subterranean caves, collapsed buildings, mountains, hazardous zones in construction, and so on.

1.4 Project Gantt Chart and Deliverables

1.4.1 Kaavish I

1. Adopting the existing hardware and making the robot walk on flat ground. This includes:
 - Re-assembling the robot to ensure proper orientation of motors. The robot was assembled by someone from EE 2023, and we cannot simply trust that it was assembled properly as improper assembly can result in having invalid commands to the motors causing irreparable damage
 - Interfacing the motor controller with a Raspberry Pi 4. The current microcontroller for the robot is outdated and is not suitable for the tasks we intend to execute on the robot. As such, we will replace it with a Raspberry Pi 4 and interface that with the existing motor controller
 - Integrating all the hardware all together and implementing a basic gait. This gait does not need to be able to deal with rough terrain, it will simply be obtained from the kinematic model of the robot and is intended to make the robot walk on flat ground
2. Building the simulation model for the robot as well as the simulation environment. This includes:
 - Identifying a suitable simulation environment. To ensure accurate simulation of locomotion strategies, a high fidelity physics engine like MuJoCo could be suitable; however, other considerations include integration with ROS and the ability to import custom models and environments

- Making a 3D model of the robot. The Lynxmotion website provides `sldprt` files that can be converted to `stp` files that can then be modified if needed and imported into any simulation software
 - Identifying the parameters for the lava tubes terrain and scaling them down appropriately.
 - Making a simulation environment for testing. This can be handcrafted based on the parameters identified, but procedural random generation will allow for the robot to be extensively tested in simulation with better generalization and there is some recent work that suggests this could be possible [28]
3. Extensive literature review of locomotion strategies, with some selected to be tested for the robot. The strategies will be selected based on a number of considerations, including the incorporation of feedback (open-loop vs closed-loop), novelty, and proven success. We will ensure that the selected strategies stretch across a number of paradigms, e.g. evolutionary algorithms, model-based RL, model-free RL etc.
 4. Literature review and initial implementation of multi-elevation mapping techniques. This includes:
 - Studying robot-centric mapping approaches and their application to confined space navigation [15, 47]
 - Implementing basic distance sensor data processing for multi-elevation map generation
 - Developing algorithms to identify spatial constraints from sensor data
 - Testing perception algorithms in simulation environment

1.4.2 Kaavish II

1. Modifications to the hardware to facilitate locomotion. This includes adding proprioceptive sensors to obtain feedback.
2. Implementing the identified locomotion strategies in simulation.
3. Implementing and testing the multi-elevation mapping system in simulation, integrating it with locomotion planning algorithms.
4. Building the physical test environment for the robot.

5. Deploying the locomotion strategies onto the physical robot and adjusting based on observed behavior. Locomotion controllers may need to be retrained with different constraints based on the behavior observed on the physical robot.

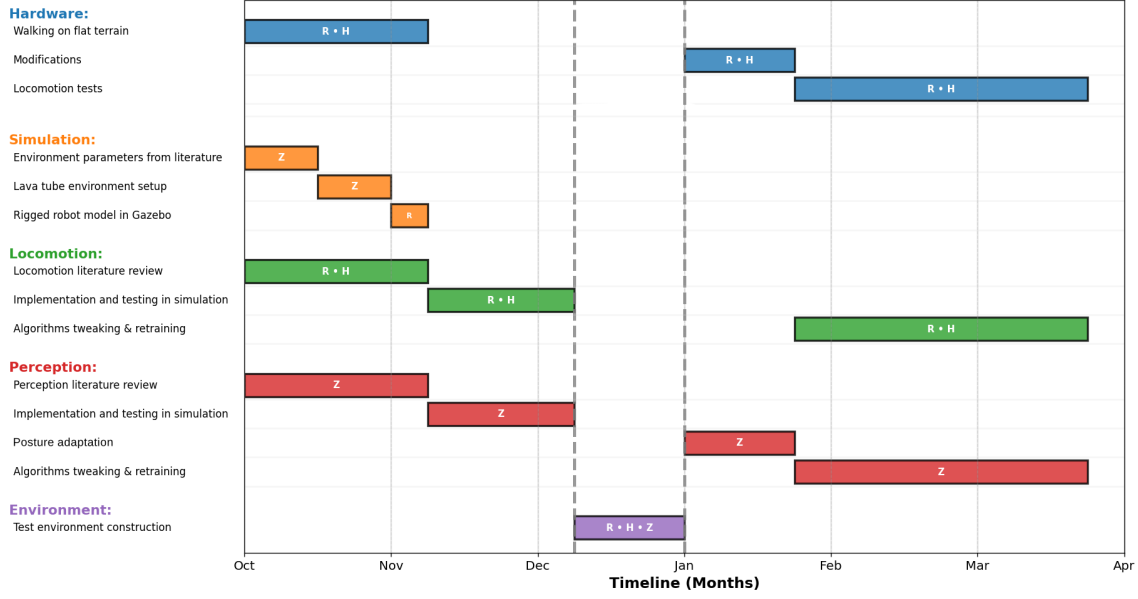


Figure 1.1: The Gantt chart for scheduling tasks per team member.

1.5 Key Challenges

The project is highly constrained by the hobby-grade hardware. Key challenges include:

- **Actuation Limits:** The motors are really weak as compared to other robots, even ones of the same size, thus making it much harder to put load on the robot or expect it to perform well. This plays a part in simulation as well as we try to accurately model the motor limits.
- **Power Constraints:** Making the robot stand draws 2A of current per side. Each power rail on our SSC-32U servo controller is rated for a maximum of 3A, leaving a critically narrow margin for dynamic movements.

- **Sim-to-Real Gap:** The robot does not have any proprioceptive sensors, so we had to retrofit an IMU and tap the potentiometers for joint state feedback. While these solutions are the most sensible out of everything we could do, this does introduce challenges as the feedback from the sensors is noisy and requires additional work to filter out noise.
- **Novelty of Posture Adaptation:** This is a significantly unexplored topic with approaches most often targetting posture adaptation for flat terrain, so concocting a personal approach deviating from literature is required here.

2. Literature Review

2.1 Robot Design

With respect to robot design, there are many factors to consider, such as the robot morphology, type of actuators, sensor suite, and compute capabilities.

Morphology refers to the physical structure of the robot as defined by the number of limbs, joints, and their kinematic alignments. For multi-legged robots, two main morphologies are commonly used: parasagittal and sprawling (see figures ?? and ??). Each of these morphologies can have a varying number of legs, with quadrupeds and hexapods being the most common. Both morphologies have their own advantages and disadvantages. Most prominently, parasagittal robots are more energy efficient, with legs straight below the hip, allowing the robot to stand without exerting torque on the joints. However, they usually have a higher center of mass and a smaller support polygon, making them more susceptible to tipping over during locomotion. On the other hand, sprawling robots, with their legs extending in the lateral direction from the hip, have a much lower center of mass and a wider support polygon, making them more stable, but less energy efficient as the joints must exert torque to hold the body up against gravity even when the robot is standing still.

Some examples of state-of-the-art parasagittal robots include the MIT Cheetah 3 [12], Boston Dynamics’ Spot [72], ETH Zurich’s SpaceBok [6], ANYbotics’ ANYmal [35], IIT’s HyQ [70], Unitree B2 [83], Unitree Go 2 [85]. All of these robots are quadrupeds having 3 DOF per leg, except for SpaceBok with 2 DOF per leg. On the other hand, examples of sprawling robots include CSIRO’s Weaver [9], CTU Prague’s HAntR [20], and Lynxmotion’s T-Hex [78], which have 5-DOF, 4-DOF, and 3-DOF legs respectively. While parasagittal quadrupeds and sprawling hexapods are the most common morphologies, other morphologies such as parasagittal hexapods, for example DOBOT’s Hexplorer [24] and RHex [2], and sprawling quadrupeds, for example the TITAN-XIII [3], have also been developed.

Beyond morphology, the actuation scheme fundamentally dictates a robot’s dynamic capabilities and its ability to negotiate rough terrain. High-performance platforms rely on torque-controlled actuators that allow the robot to act as a virtual spring, absorbing unexpected impacts from uneven ground. For example, the MIT Cheetah 3 [12] utilizes custom quasi-direct drive (QDD) actuators, which combine high-torque brushless motors with low-ratio gearboxes to achieve high mechanical transparency and backdrivability. ANYmal [35] and StarlETH [36] employ Series Elastic Actuators (SEAs), placing a physical spring between the motor and the joint

to absorb shocks and store energy, while the HyQ platforms [70] rely on hydraulic cylinders for extreme force density and impact resistance. Similarly, the industrial-grade Unitree B2 [83] utilizes high-torque-density motors capable of supporting up to 120 kg payloads over severe terrain. To bridge the gap to more accessible hardware, open-source research platforms like PADWQ [39] achieve similar dynamic transparency by leveraging custom low-gear-ratio transmissions and 3D-printed components. Conversely, low-cost and educational platforms, such as the Stanford Pupper [89], HAntR [20], and our T-Hex kit [78], utilize position-controlled hobby servos. While affordable and lightweight, these rigid actuators lack force feedback (and often, any kind of feedback) and cannot naturally absorb shocks, requiring specialized algorithms to detect and react to contacts.

To traverse rough terrain and assess spatial constraints, the robot needs a suite of sensors, which is generally divided into proprioceptive and exteroceptive sensors. Proprioceptive sensors measure the robot’s internal state; almost all modern legged robots feature high-resolution joint encoders to measure joint positions, torque sensors to measure joint efforts, and a central Inertial Measurement Unit (IMU) placed near the Center of Mass (CoM) to estimate body orientation, position, acceleration, and linear and angular velocity. Contact sensors, detecting when the feet make and break contact with the ground, are also common. Relying heavily on this high-frequency proprioceptive feedback, platforms such as the Ghost Robotics Vision 60 [86] emphasize blind locomotion to traverse tall grass or deep water where cameras might fail. Similarly, the MIT Cheetah 3 [12] utilizes proprioception and robust sensor fusion [11] to run and gallop at high speeds, bounding up stairs and traversing over unstructured terrain completely blind. Highly specialized robots like SpaceBok [6] also rely heavily on internal state estimation, integrating precise IMUs and leg kinematics to achieve in-flight attitude control in low-gravity environments.

However, exteroceptive sensors, which measure the outside world, are critical for posture adaptation in confined spaces. State-of-the-art robots like Spot [72, 13] and ANYmal [29] feature an array of stereo depth cameras placed around the perimeter of the chassis to provide 360-degree close-range obstacle detection, often coupled with a 3D LiDAR at the top of the chassis for long-range spatial mapping. Other platforms achieve environmental awareness through more directional sensing. To achieve similar perceptive capabilities on a budget, platforms like the Unitree Go2 [85] and PADWQ [39] mount a single forward-facing depth camera (such as an Intel RealSense) or a solid-state LiDAR whereas Weaver [9] utilizes stereo cameras to perceive the terrain.

Finally, running advanced locomotion algorithms (the kinds of which we will discuss in the next section) that take in sensor data and output joint commands

requires significant onboard compute capabilities, typically arranged in a hierarchical architecture. The low-level control loop, which is mainly responsible for the actual locomotion itself, that is estimating states, calculating joint commands to move and maintain balance, must run at high frequencies (often between 50 Hz and 1000 Hz) and is usually handled by a dedicated real-time microcomputer. The MIT Cheetah 3 [12] runs its purely proprioceptive locomotion controller at 1000Hz using a dedicated onboard x86 PC running a real-time operating system. For systems with perception modules, high-level planning tasks, such as processing dense point clouds to localize the robot, assess spatial constraints, build a map, or plan a trajectory run at much lower frequencies (5 Hz to 50 Hz), mainly due to computational expenses, but also due to assumptions regarding how fast the perception data realistically changes. ANYmal [35] employs multiple onboard Intel Core i7 processors, dedicating one exclusively to real-time locomotion control and another entirely to perceptive navigation and mapping [32]. PADWQ [39] explicitly incorporates an onboard GPU to handle its perceptive locomotion.

Considering its sprawling morphology, position-controlled servos with less than 1 Nm peak torque, no integrated feedback, and a low-speed servo controller, the T-Hex, especially with two legs removed, is greatly limited by its hardware as compared to other robots in the similar weight class, requiring us to augment it with the bare-minimum sensors for robust locomotion, and work around the limitations of its actuators and servo controller. Algorithms that require minimal state feedback, such as joint state and body orientation, and can run at low frequencies are most suitable for the T-Hex. This motivates an extensive literature review of locomotion approaches to find the most suitable ones for our hardware.

2.2 Locomotion and Perception

The locomotion of legged robots in various challenging scenarios has been an active area of research for the past few decades. Locomotion controllers for legged robots can be divided into two-main categories: model-free controllers and model-based controllers. Here, the model being referred to is the dynamics model of the robot, which is typically used to relate the forces and torques on the robot’s legs to the motion of the robot body[12, 22].

One of the simplest model-free control techniques to make a legged robot walk is to use an open-loop¹ kinematic gait that has each leg follow a pre-computed foot trajectory[75]. A controller like this is easy to implement thanks to well-established approaches for deriving legs’ kinematic equations, but is expected to perform poorly

¹As opposed to closed-loop, which means to incorporate state feedback.

in anything but flat terrain. A legged robot is a highly dynamic system, and such a controller ignores the dynamics entirely. Furthermore, not incorporating feedback means the controller is blind to when a leg doesn’t reach the desired position. If the leg hits an obstacle in the middle of its swing phase blocking its movement, the leg will continue to push against the obstacle, instead of adjusting its trajectory, like by increasing the amplitude of the swing, to overcome it. These factors, combined with the reliance on good gait parameters such as swing amplitude, stride length, and the lateral distance from the hip², makes such a controller unsuitable for unstructured terrain.

Central Pattern Generators (CPGs), inspired by biological neural networks in animal nervous systems, are used in more sophisticated model-free controllers to generate kinematic references for the robot’s joints to follow. The implementation of CPGs varies across literature [56, 7, 27, 5, 18, 57, 91], but at a high-level, a CPG is attached to each joint in the robot and generates oscillatory signals that act as reference joint trajectories. CPGs in each joint can communicate with each other implicitly through mathematical coupling, which allows for designing specific gaits that rely on phase differences between groups of legs. Performance of CPGs also relies on the choice of parameters, and learning-based methods or derivative-free optimization techniques like evolutionary algorithms are often used to find the best set of parameters by simulating the robot in virtual terrain [44, 64, 25]. When state feedback is incorporated into such controllers, they can be surprisingly robust even in non-flat terrain involving slopes, stairs, and unstructured elements such as pebbles and stones [5, 7, 18].

Recently, Deep Reinforcement Learning (DRL) has been under the spotlight for learning robust locomotion controllers for legged robots. Proximal Policy Optimization (PPO), a policy-gradient method developed at OpenAI [69], is used when training DRL-based controllers due to its impressive performance particularly for tasks related to robot locomotion and manipulation. In PPO, two separate neural networks are trained simultaneously. The value network is responsible for learning how good each state is, whereas the policy network learns the best actions to take at each state - only the latter is actually used after training. To train the two networks, many instances of robots are simulated in virtual environments [67]. NVIDIA’s Isaac Lab [53] provides pre-developed plug-and-play packages for training the locomotion of any legged robot in a variety of environments. To minimize the sim-to-real gap, domain randomization is used to simulate unmodeled factors and wear and tear of hardware, which makes the controller robust to external disturbances. DRL-based

²This is only applicable for robots with a sprawling morphology, like T-Hex. For parasagittal robots, such as Spot or ANYmal, the lateral distance from the hip can be 0.

locomotion controllers are successful in a wide variety of terrains and demonstrate impressive dynamic movement abilities [79, 47, 43, 19, 46, 80, 66, 65, 50, 40].

Model-based approaches require some dynamic model of the robot. A perfectly accurate dynamic model is usually too complex to be directly used; however, under specific assumptions, a simplified dynamic model can be used to approximate the robot’s dynamics.

Among the first model-based approaches to legged locomotion used the Simple Linear Inverted Pendulum (SLIP) model, first proposed by Cavagna [16] for modeling locomotion of animals. The SLIP model can be visualized as a point mass on a spring that is passive and conservative, representing the body and leg respectively. With slight modifications, the SLIP model has been used to design impressive locomotion controllers robust to disturbances [63, 68, 59]. However, the SLIP model isn’t directly applicable to all morphologies of robots, including sprawling legged robots such as T-Hex. Furthermore, such approaches usually rely on instantaneous analytical solutions which requires making strict assumptions regarding the motion of the robot, such as assuming a contact schedule for legs, which can be violated in unstructured terrain.

Modern approaches move away from analytical solutions to optimization-based controllers and use more complex and accurate dynamic models [12, 22, 42, 34, 55, 30, 58, 10, 23, 51, 87, 31, 60, 36, 88, 41, 45, 81, 17]. Such controllers plan a center of mass trajectory, typically based on user commands of linear and angular velocity of the body, and then optimize for ground reaction forces exerted by the stance legs to enforce the desired dynamics on the robot’s center of mass, such as zero roll and pitch velocity to remain upright or zero vertical acceleration to maintain a base height. Swing legs are typically commanded to follow a reference swing trajectory as is typical with other types of locomotion controllers.

However, the robot also has to consider how to adjust itself in order to minimize collisions with the walls and ceiling in confined environments. For that, it requires a way of sensing the environment that mere proprioceptive sensors cannot provide. Exteroceptive sensors, such as RGB-D cameras and LiDARs, that provide point clouds containing depth information of the surrounding terrain prove to be a necessity. To understand the environment’s geometry, one proposed method is to split the pointcloud into patches and fit local planes using covariance eigenanalysis, yielding slope and roughness [4]. This can be expanded upon to include per-cell uncertainty and height leading to a probabilistic elevation map [26, 1]. However, these elevation maps do not consider ceilings or overhangs, which introduce multiple z-values. To solve this, the OctoMap [33] provides full 3D occupancy for the entire space instead of estimating surfaces. On the other hand, Voxblox [54] utilizes signed distance fields (SDFs) to allow clearance or available space to be obtained efficiently. From there,

traversability; that is, a metric determining safer spots for the robot to go to; is computed. This could range from determining rules for desired slope, roughness, and clearance to learning foothold quality via a network and robot experience. Now, given traversability, the robot must choose how to adapt its configuration or posture to fit in; this can be achieved via abstracting the robot into a deformable bounding box and creating a trajectory of desired body postures [15, 14]. Perception-based locomotion has also seen significant integration in RL as demonstrated by Miki et al. who used elevation maps as inputs into RL policy [47]. These were replaced with full 3D volumetric representation for confined spaces [48].

3. Software Requirement Specification (SRS)

The project is divided into three core streams:

- **Locomotion:** Developing capability of the robot to traverse rough terrain
- **Perception:** Developing capability of the robot to sense its environment and take decisions, e.g. with regards to posture adaptation
- **Simulation:** Developing an accurate virtual environment to safely implement, test, and iterate over the locomotion and perception strategies before hardware deployment

3.1 Functional Requirements

3.1.1 Simulation

- The simulation system must allow for randomly generating environments
- The simulated environments must have rough terrain
- The simulated environments must have narrow passages, areas with low ceilings, and protrusions from the ground to test posture adaptation
- The generation process must include parameters for adjusting roughness in terrain, ground friction, range of width and height of passages, and range of size of protrusions
- The simulation system must support physics
- The T-Hex robot must be simulated with user-defined mass properties

3.1.2 Perception

- The robot must be integrated with a camera to be attached on the robot in simulation
- The robot must be able to build a map of the immediate environment in simulation, i.e., what is directly ahead of the robot

- The robot must be able to use a map of the immediate environment in simulation to make decisions to switch to a narrower posture to pass through thin passages, a shorter posture to pass under low ceilings, or a posture with higher body clearance to pass over protrusions

3.1.3 Locomotion

- The robot, in both simulation and hardware, should be able to walk on flat and rough terrain
- The robot, in both simulation and hardware, should be able to adapt its posture in both flat and rough terrains
- The robot, in both simulation and hardware, must be teleoperable - it should take a target position relative to itself as input and execute a straight line trajectory to reach it, adjusting to the terrain and obstacles

3.2 Non-Functional Requirements

- The robot should be able to walk in terrains with local vertical step size up to 8cm
- The robot should be able to walk in terrains with local slope up to 15 degrees
- The robot should be able to pass through passages as narrow as 38cm in simulation
- The robot should be able to pass through areas with ceilings as low as 16cm in simulation
- The robot should be able to pass over protrusions of up to 6cm in simulation
- The robot should be able to walk in terrains with friction coefficient of 0.5-0.7
- The robot should be able to walk in terrains with sand, ash, and pebbles as big as 1cm
- There must be at least three “postures” that the robot should be able to move in: a shorter posture, a regular posture, and a posture with higher body clearance in simulation

- For data integrity purposes, there must be a Docker image containing everything necessary to run the simulation of the robot in randomly generated environments
- The entire codebase must be well-commented with citations included in either the README or as comments in the file themselves for code that is taken or adopted from someplace else
- The entire codebase must be well-structured and maintained with complete version control on GitHub to allow for backups in case of catastrophe
- Documents detailing the robot’s hardware architecture and its locomotion and perception strategy must be prepared

3.3 Use Cases

We have only one user: the Operator. For that, the following use cases are defined:

- **Commands:** The operator gives position commands relative to the robot
- **Adjust simulation parameters:** The operator is able to adjust the parameters of the simulation environment
- **Generate simulation environment:** The operator is able to generate simulation environments
- **Pick and place simulated robot:** The operator is able to move the simulated robot by picking it up and placing it at the desired position

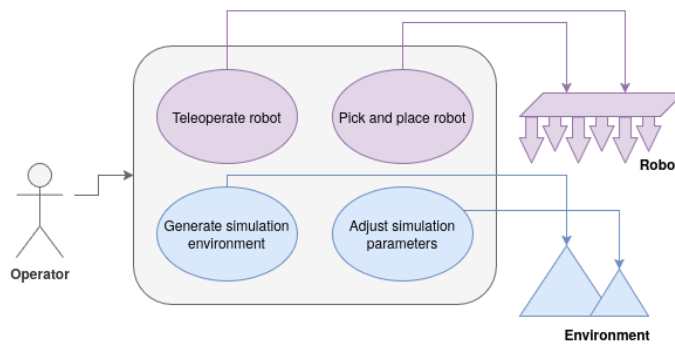


Figure 3.1: Use Case Diagram for the Legged Robot System.

3.4 Block Diagrams

3.4.1 Perception

In simulation, the camera takes in the environment and processes it as RGB-D data. It also possesses the IMU data which will tell about the robot's state. Both of these are used to create an elevation map for the robot to not only know where it is in space, but also how to navigate its environment as well as obstacles. There are converted to a signed distance field; that is, a field determining where is safe for the robot to proceed without collision; which lead to vertical and lateral clearance that serve as the basis for posture adaptation. This outputs a posture adaptation signal determining a desired height for the robot, which is sent to the locomotion algorithm. (The width is inferred from the height as described later.)

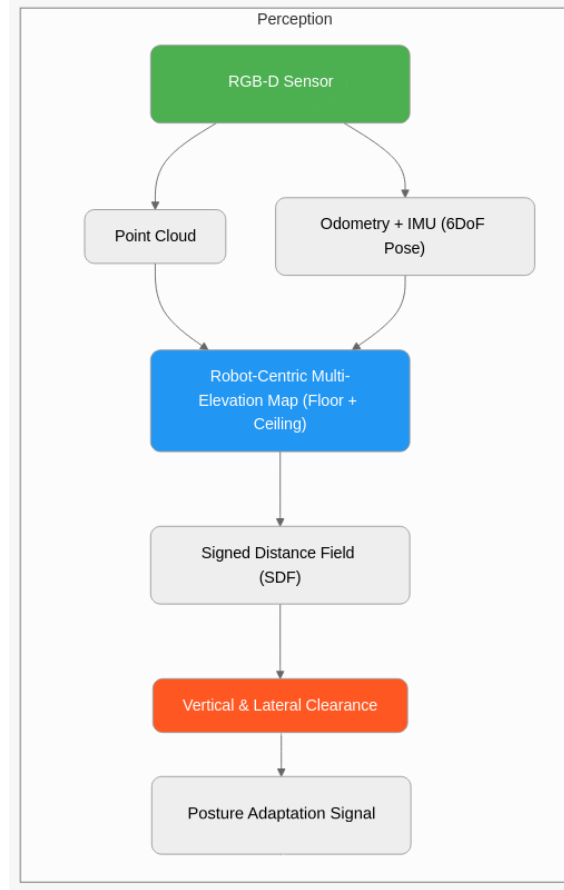


Figure 3.2: Perception Diagram for the Legged Robot System.

3.4.2 Locomotion

The first of the three locomotion algorithms we have covered is the simplest: the kinematic gait. This operates on a gait schedule, which defines stance and swing legs with their own trajectories. Once generated, the trajectories are tracked before sent as motor commands.

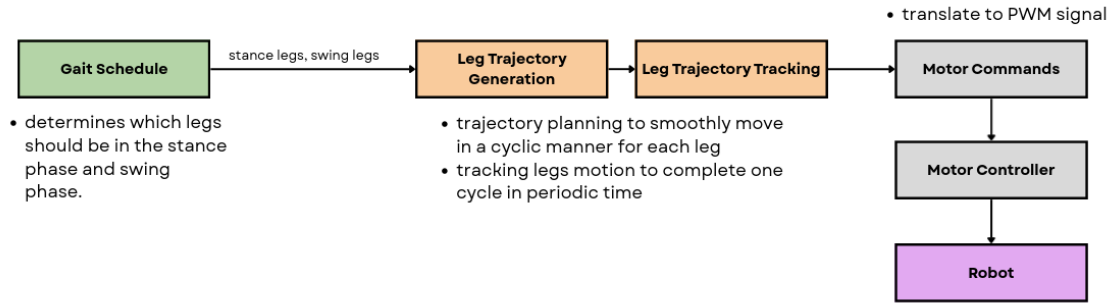


Figure 3.3: Kinematic Locomotion Diagram for the Legged Robot System.

The next diagram represents the general locomotion architecture that is observed in literature. Based on user commands and the persistent gait schedule, a trajectory for the robot’s center of mass is planned, which is used to solve for the forces exerted by the legs in stance (i.e., in contact with the ground) and to generate the trajectories to be generated by the legs in swing. These forces and desired positions are converted into motor commands that are sent to some motor controller and then to the motors. The robot’s states (joint positions, roll, pitch, yaw) are incorporated at different steps to ensure robustness.

A third and significant locomotion architecture is that of reinforcement learning (RL). In this case, the policy network starts with user commands and passes its predicted movements as motor commands. Similarly to control, the state estimation segment sends feedback to the network, which adjusts itself accordingly and repeats the cycle. In our case, this particular paradigm’s flavor is mixed with a deep learning network, thus being deep reinforcement learning (DRL).

3.4.3 Hardware

Our system contains hardware modules depicted in the diagram above. The micro-controller receives user commands from the user’s laptop, IMU data from the IMU, and estimates of the robot legs’ joint angles from some state estimation method (e.g.,

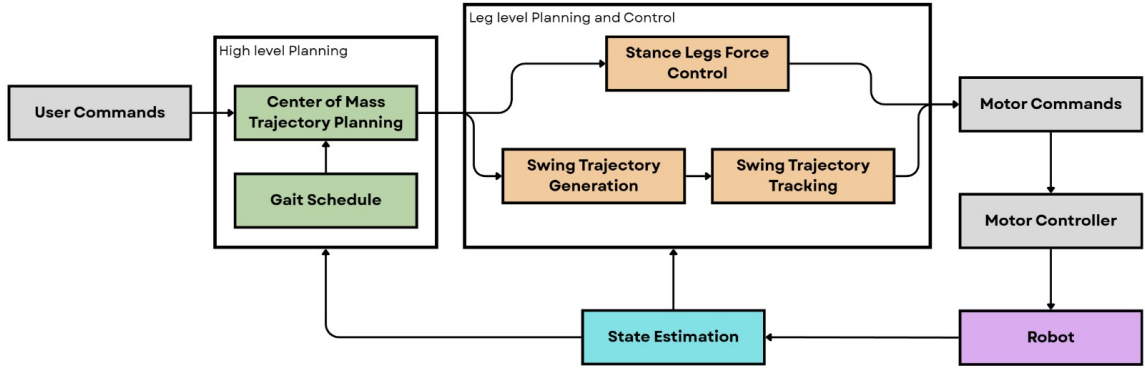


Figure 3.4: Dynamic Locomotion Diagram for the Legged Robot System.

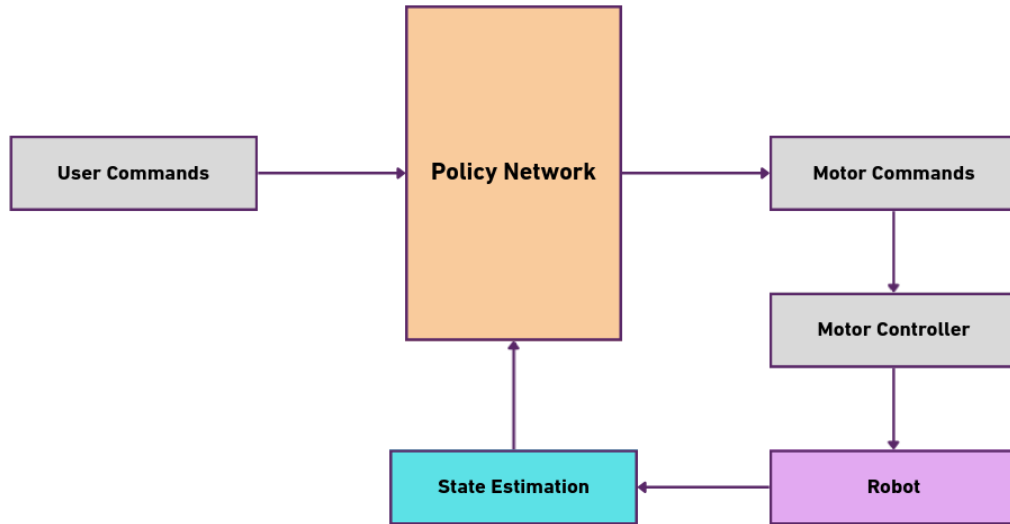


Figure 3.5: DRL Locomotion Diagram for the Legged Robot System.

motor encoders or motion capture cameras). The microcontroller runs the locomotion algorithms and outputs desired motor angles to the separate motor controller. The motor controller then generates pulse width modulation (PWM) signals that are received by the motors.

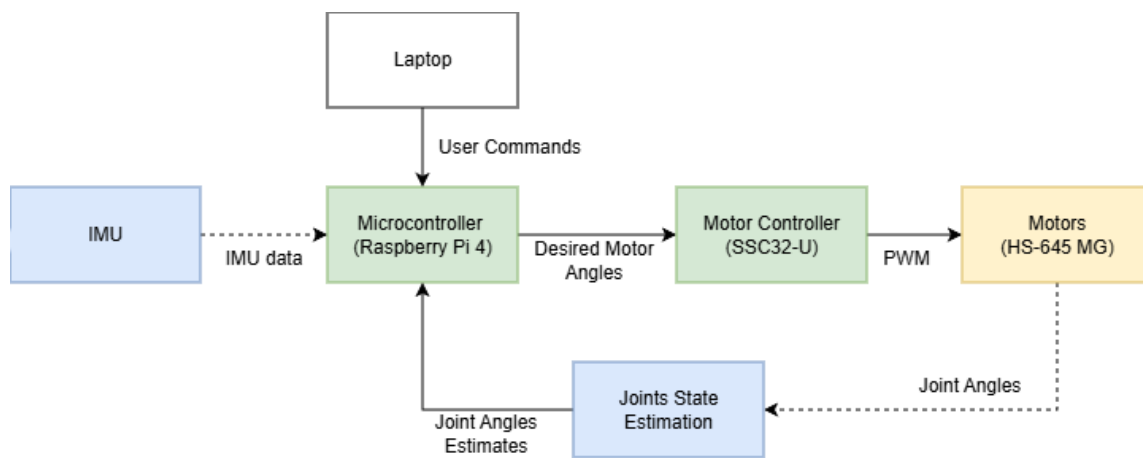


Figure 3.6: Hardware Diagram for the Legged Robot System. Lines indicate the flow of data, with dotted lines indicating the robot states or world states being observed

4. Software Design Specification (SDS)

4.1 Software Design

With regards to our project, we shall consider our simulation as software for the SDS. Hence, this subsection explains the overall design of the simulation system including a UML class diagram.

This particular system is structured around three core modules: **Environment**, **Perception**, and **Locomotion**. These modules interact to allow for teleoperating the robot with posture adaptation on rough terrain.

Figure 4.1 shows the UML class diagram.

The Operator acts as the user interface where the user teleoperates the robot. They also have the liberty to adjust the parameters of the environment such as terrain roughness and friction. The SimulationEnvironment is where the testing happens. It is reliant on the mentioned parameters and generates the environment to show to the physics simulator (Gazebo). Then, the SimulatedRobot is placed within the scene. The Robot is the T-Hex in simulation and manages state information while executing commanded trajectories. It aggregates both the LocomotionController and PerceptionModule; the former manages gaits and trajectories through Ground Reaction Forces for legs in stance and smooth swing leg trajectories to send to the MotorController which executes the commands, and the latter is where the robot senses data from the environment via RGB-D and IMU readings to construct the elevation map. This map is used to calculate clearance; that is, collision-free space; through a signed distance field. This is used to adapt the robot's posture based on the vertical and lateral clearance.

4.2 Data Design

While we do not need persistent data and hence a database, it is worth discussing data structures used in the project. All data is managed within simulation and Docker environment. This data flows through Figures 4.2 and 4.3, and the hardware data path in a physical build is shown in Figure 4.4.

The most important data structure is the Elevation Map. It is a 2D grid entity timestamped for accuracy meant to store terrain heights. From here is derived the Signed Distance Field (SDF) which models clearance by retrieving distance values to obstacles.

The robot's behavioral data relies on Posture Parameters, a structure defining modes using attributes namely leg spread, fixed body height, etc. On the other hand,

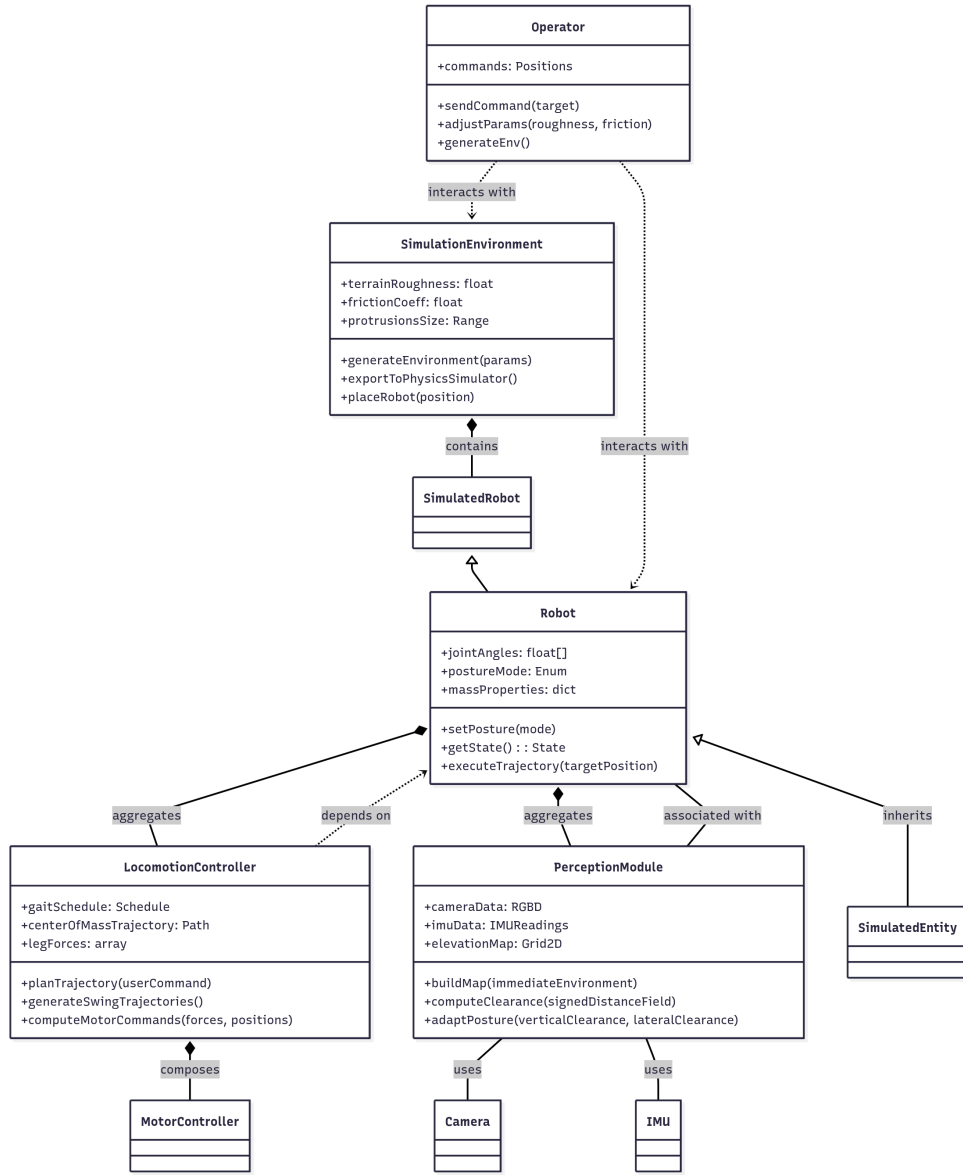


Figure 4.1: Class Diagram for the Legged Robot System.

the environment relies on Simulation Parameter which are roughness, friction range, maximum protrusion size (6cm), etc. For brevity's sake, we shall not be including the kinematic and DRL diagrams here.

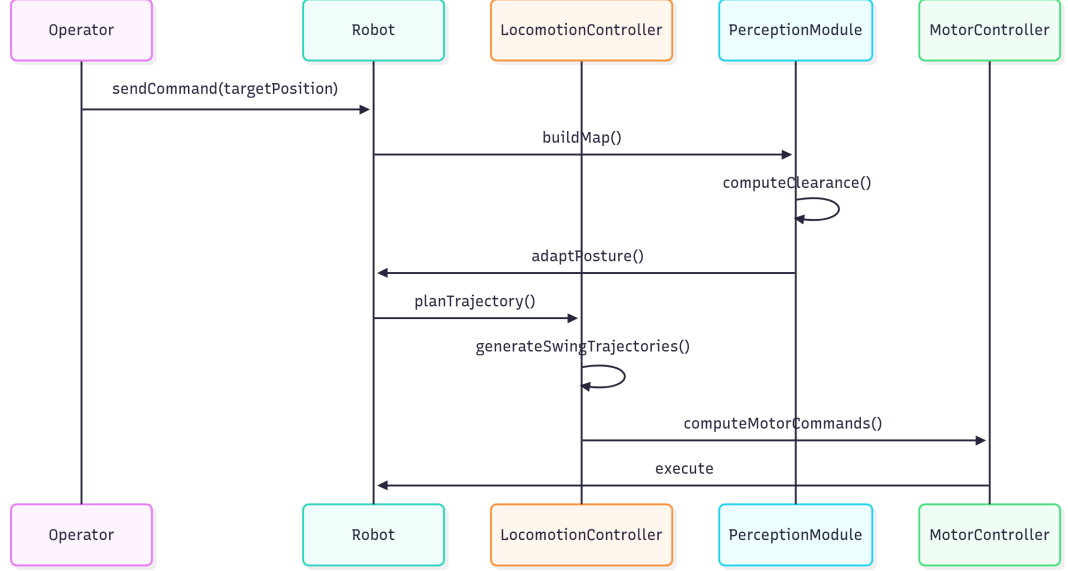


Figure 4.2: Sequence Diagram for the Locomotion Pipeline, detailing the interaction between the operator, robot, and core control modules.

4.3 Technical Details

Here, we discuss algorithms. Further information is provided in the Robot Technical Document.

The Trajectory Planning Algorithm handles movement. Using Model Predictive Control (MPC), it processes user commands and robot state, returning Center-of-Mass trajectories, leg forces, and swing trajectories. This is to traverse rough terrain.

Next, the Map Building Algorithm is to build the map from raw RGB-D camera data and IMU readings, applying point cloud processing and odometry correction to generate a 2D elevation map for finding clearance. Essentially, points are classified to floor and ceiling based on their position with regards to the camera frame.

The Posture Adaptation Algorithm reads the elevation map and signed distance field to dynamically adjust the posture:

Shorter: For low ceilings.

Narrower: For thin gaps.

Both: Stop. Else no change from previous state.

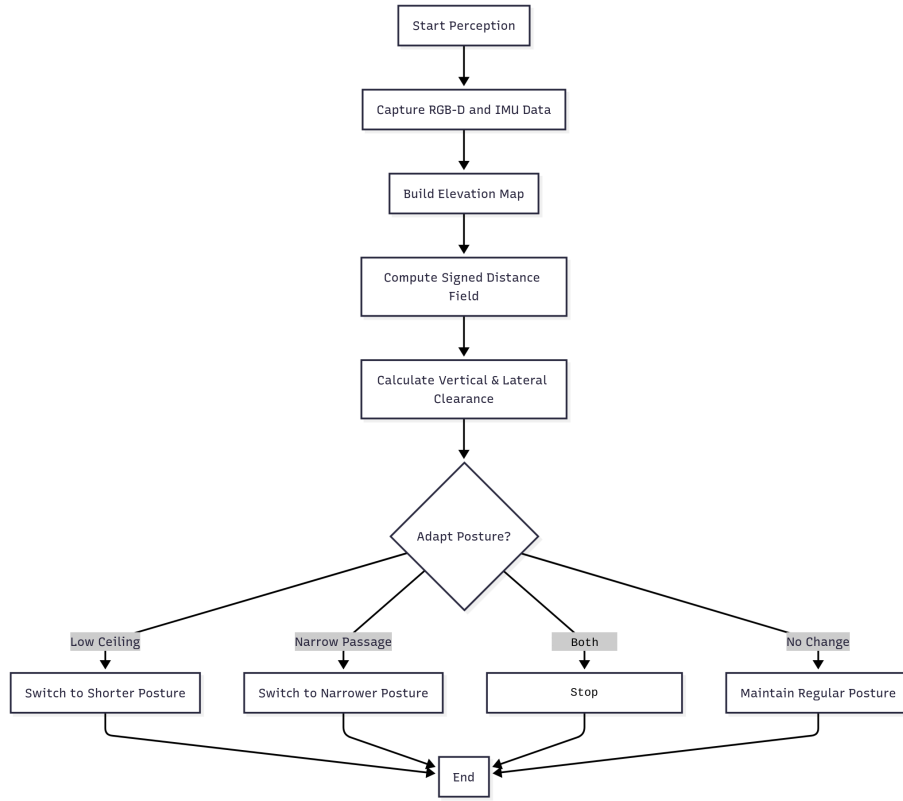


Figure 4.3: Flowchart for the Perception Pipeline, illustrating the steps from data capture to posture adaptation decision.

The Environment Generation Algorithm uses random procedural generation based on user parameters aforementioned to create rough terrains for simulation.

The microcontroller converts movement commands from locomotion into final PWM signals sent directly to the motors.

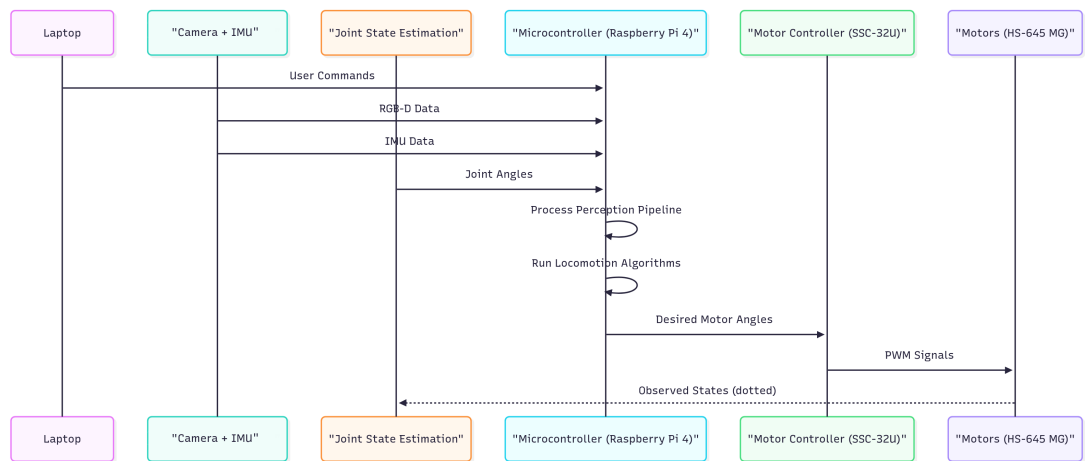


Figure 4.4: Sequence Diagram for the Hardware Pipeline, showing the flow of data and commands between physical components.

5. Methodology, Experiments and Results

5.1 Methodology and Experiments

The T-Hex kit, as the name suggests, is originally a hexapod. However, due to hardware failures present in our kit, the two middle legs had to be taken out, making it a quadruped. As we wish to eventually extend this work on hardware, we treated the robot as a quadruped. Going forth, T-Hex refers to this quadruped version of the original kit as seen in figure 5.1.

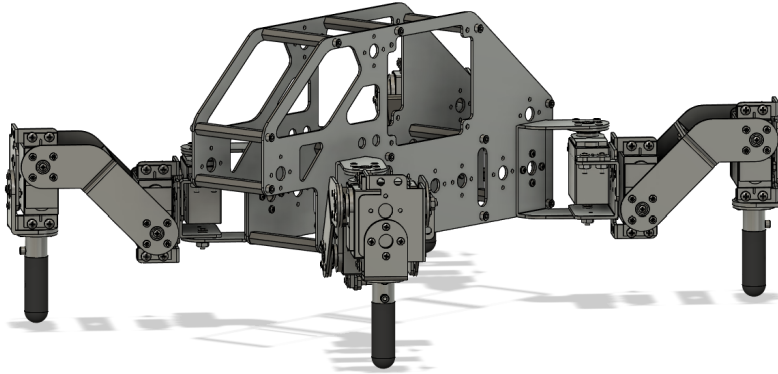


Figure 5.1: T-Hex (as seen in SolidWorks) with its middle legs removed, making it a quadruped.

5.1.1 Kinematics

The motors of the T-Hex take position commands, that is what angle they should move to, as opposed to torque commands which are common in high-end motors [12, 35]. Each leg of T-Hex has 3 motors (or joints), at the hip, knee, and ankle. Many locomotion controllers, such as an open-loop kinematic gait, output the position of the foot, that is the end point of the legs, at any given time. The problem of determining what joint positions result in a desired foot position is known as the Inverse Kinematics (IK) problem. Solving the IK of a legged robot is well-established. Using the Denavit-Hartenberg (DH) convention, frames of reference are assigned across the robot's legs and the homogenous transform matrices can be multiplied to obtain equations that take in joint positions and output resulting foot

positions, known as the Forward Kinematics (FK) equations. The FK equations can then be used to derive the IK equations, that take in foot positions and output the corresponding joint angles. A detailed derivation is omitted for the sake of brevity, but a similar derivation can be found in [75]. The IK equations for our robot are as follows:

$$\theta_1 = \text{atan2}(y, x) \quad (5.1)$$

$$\theta_2 = \begin{cases} \arcsin\left(\frac{z}{a_2}\right) + \phi_2 & \text{for right legs} \\ \arccos\left(\frac{z}{a_2}\right) - \phi_2 & \text{for left legs} \end{cases} \quad (5.2)$$

$$\theta_3 = \begin{cases} \frac{\pi}{4} & \text{for right legs} \\ -\frac{\pi}{4} & \text{for left legs} \end{cases} \quad (5.3)$$

$$\theta_4 = \begin{cases} \phi_1 - \arcsin\left(\frac{\Psi}{a_1}\right) & \text{for right legs} \\ -\arccos\left(\frac{\Psi}{a_1}\right) - \phi_1 & \text{for left legs} \end{cases} \quad (5.4)$$

where Ψ , a_1 , $\phi_1 a_2$, and ϕ_2 are defined as:

$$\Psi = \frac{1}{2L_4} \left[(x \cos \theta_1 + y \sin \theta_1 - L_1)^2 + z^2 - L_2^2 - L_3^2 - L_4^2 - 2L_2L_3 \cos \theta_3 \right] \quad (5.5)$$

$$A_1 = L_2 \cos \theta_3 + L_3 \quad (5.6)$$

$$B_1 = L_2 \sin \theta_3 \quad (5.7)$$

$$a_1 = \sqrt{A_1^2 + B_1^2} \quad (5.8)$$

$$\phi_1 = \begin{cases} \text{atan2}(A_1, B_1) & \text{for right legs} \\ \text{atan2}(B_1, A_1) & \text{for left legs} \end{cases} \quad (5.9)$$

$$A_2 = L_2 + L_3 \cos \theta_3 + L_4 \cos(\theta_3 + \theta_4) \quad (5.10)$$

$$B_2 = L_4 \sin(\theta_3 + \theta_4) + L_3 \sin \theta_3 \quad (5.11)$$

$$a_2 = \sqrt{A_2^2 + B_2^2} \quad (5.12)$$

$$\phi_2 = \begin{cases} \text{atan2}(B_2, A_2) & \text{for right legs} \\ \text{atan2}(A_2, B_2) & \text{for left legs} \end{cases} \quad (5.13)$$

where x , y , and z are input foot coordinates in the *hip frame* of the leg and L_i indicates the length of the i -th link. While each leg in the T-Hex has only 3 motors,

it has a 45 degree bend in between the knee and ankle motors (as can be seen in figure 5.1) which we model as a rigid joint (see eq. (3)); hence, we have four links.

5.1.2 Simulation

To simulate the robot, we used the available CAD model (as seen in figure 5.1) and the SolidWorks plugin SW2URDF [77] to export a URDF file for simulating the robot. Certain properties, such as mass and inertia, were exported incorrectly and had to be manually adjusted using calculations from SolidWorks. Furthermore, we also made a simplified collision mesh to be used in place of the high fidelity CAD model to speed up simulation. We used Gazebo Sim as our primary simulator.

To simulate terrain, we had two types of terrains: a flat world with some narrow passageways and crawl spaces made out of cuboids, and a rough terrain cave procedurally generated via modifying PLUME [28], a procedural cave generation framework, for our lava tube case. In both terrains, the ground’s friction coefficient was set to 0.7.

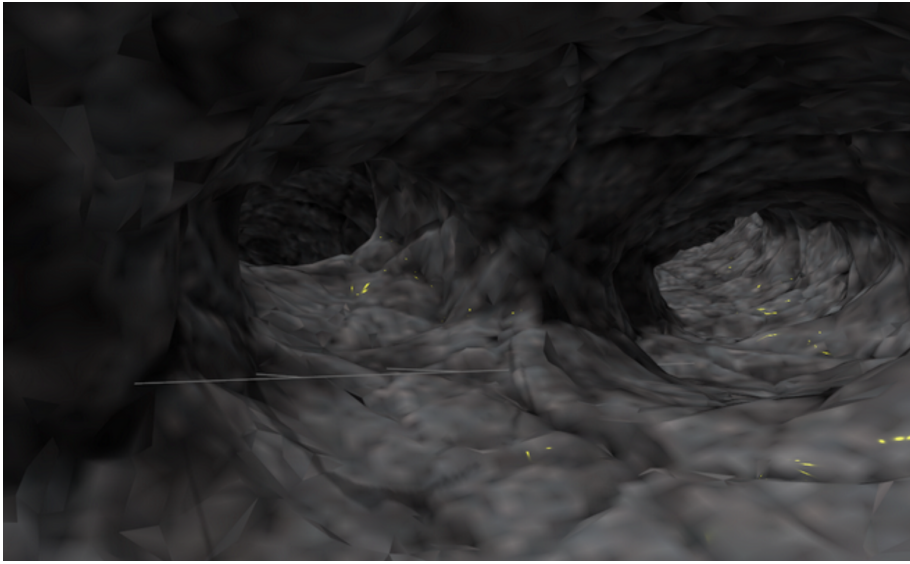


Figure 5.2: Interior of cave generated using PLUME [28]

After simulating the robot and terrains to test our locomotion controllers in, we set out to work on the locomotion controllers themselves. To cover both model-free and model-based controllers, we worked on the following three controllers:

1. An **open-loop kinematic gait** to serve as a baseline for walking in flat terrain

2. A **deep reinforcement learning** based controller trained in Isaac Lab
3. A **model-based controller** based off of MIT Cheetah 3’s locomotion controller [12]

In simulation, the work on perception was taken separately, with the goal of giving posture signals that could be sent to the above controllers.

5.1.3 Open-loop kinematic gait

For this controller, we use a crawl gait. For a quadruped, a crawl gait is one where at most one leg is in swing phase at a given time, giving a wider support polygon under the robot with three legs in stance. Hence, the stance phase is three times longer than the swing phase. The different permutations for the swing legs may result in better or worse stability. Typically, alternating sides (left and right) in the swing order helps keep the robot from tilting too far on one side. A possible schedule is given in table 5.1.

Table 5.1: Crawl Gait Schedule

Leg	$0 \leq t < \frac{N}{4}$	$\frac{N}{4} \leq t < \frac{N}{2}$	$\frac{N}{2} \leq t < \frac{3N}{4}$	$\frac{3N}{4} \leq t < N$
FR	Swing	Stance	Stance	Stance
BL	Stance	Swing	Stance	Stance
BR	Stance	Stance	Swing	Stance
FL	Stance	Stance	Stance	Swing

We pre-compute trajectories of N discrete foot positions for each leg. A normalized phase variable $\tau \in [0, 1]$ dictates how far along a leg is in either swing or stance phase (based on the gait schedule). In swing phase, a quadratic bezier curve is used to generate trajectory points whereas in stance phase, linear interpolation between the start and end points of the swing phase can be used.

$$\mathbf{p}_{swing}(\tau) = \begin{bmatrix} X \\ (1 - \tau)^2 P_y + 2(1 - \tau)\tau Q_y + \tau^2 R_y \\ (1 - \tau)^2 P_z + 2(1 - \tau)\tau Q_z + \tau^2 R_z \end{bmatrix} \quad (5.14)$$

$$\mathbf{p}_{stance}(\tau) = \begin{bmatrix} X \\ \frac{T}{2}(1 - 2\tau) \\ S \end{bmatrix} \quad (5.15)$$

where X is the longitudinal distance from the hip (the sprawl of the robot), S is the vertical distance from the hip (how far above the ground the robot stands), T is the total distance covered in the stance phase, and $P = [X, -T/2, S]^T$, $Q = [X, 0, S + 2A]^T$, and $R = [X, T/2, S]^T$ are the control points of the bezier curve, where A is the amplitude of the swing phase (how far above the ground is the apex of the swing). To make the generated trajectories parallel to the length of the robot, the trajectories for each leg were rotated by the angle corresponding to the offset of the corresponding hip frame in zero position (the hip motor at zero degrees) from the frame placed at the center of the robot.

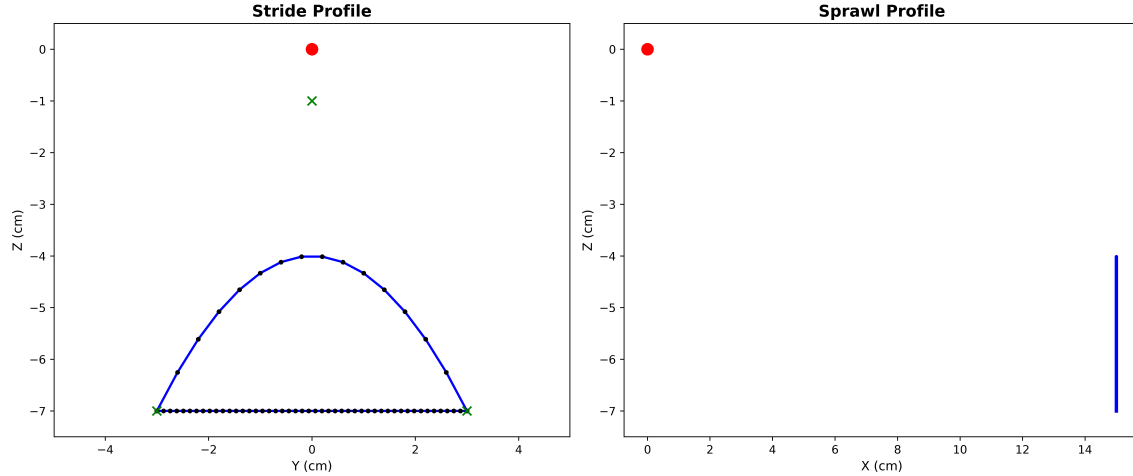


Figure 5.3: Visualization of Bezier Curve with $N = 64$, $X = 15$, $S = -7$, $A = 3$, $T = 6$ in the hip’s Y-Z plane (left) and X-Z plane (right). The red point indicates the location of where the hip frame was placed. The green crosses (in the left plot) indicate the bezier curve control points P , Q , and R .

5.1.4 Deep Reinforcement Learning-based Controller

To train a DRL-based controller, we used Isaac Sim to convert the robot’s URDF into an USD file. Then, we used a pre-existing Isaac Lab setup for velocity tracking for ANYmal C and modified it for our use case. 4096 instances of T-Hex¹ were spawned in an environment containing three different categories of terrains: random noise rough terrain created using grids of points of varying heights connected together with planes, random blocks rough terrain consisting of flat blocks of varying

¹Other hyperparameters were kept the same as [67]

heights arranged in a grid, and random inclines created by taking the maximum and minimum of planes of varying slopes. The maximum vertical distance between any two points in the random noise rough terrain was set to 6cm, the maximum vertical distance between adjacent blocks in random blocks rough terrain was set to 3cm, and the slope of the planes in random inclines was between -15 and 15 degrees. However, as is typical in literature [67, 65, 32], instead of spawning all robots at the highest end of the difficulty for each category, a curriculum learning approach was adopted where the robots would spawn in easier difficulty variants of each category, and advance to higher difficulties after successfully performing at the lower difficulty. Further details regarding the setup can be found in [67].

The input to the policy network was the robot’s current joint positions, the projected gravity vector (the gravity vector in robot’s base frame), its angular velocity, previous actions given by the policy, and the user commands for xy -linear velocity and z -angular velocity (yaw rate). The network’s output was joint commands for each of the twelve joints. The value network was given all the same inputs as the policy network, along with some privileged information to speed up training, such as joint velocities, robot’s base linear velocity, and the height values of 128 random points on the terrain around the robot.

Table 5.2 lists the rewards and penalties used during training along with their weights.

5.1.5 MIT Cheetah 3-like Controller

Heavily inspired by the MIT Cheetah [12] control architecture and its implementations [49, 71, 76], we have implemented a similar controller, albeit with some changes according to our structure and limitations.

At the highest level, the user gives commands for desired linear and angular velocity of the robot’s base. The controller is responsible for generating torque commands for the stance and swing legs, which are decided base on the gait schedule.

For the gait schedule, we implemented the crawl gait implementation taken from [71] which publishes the nominal schedule for all legs, along with the swing and stance times for each leg and how much of each state has been completed by a leg. This information, along with state feedback, is passed to an FSM which determines early contacts only, as opposed to [12] determining both early and late contacts. The current state of each leg is published and this information is utilized by the stance and swing controllers.

The stance controller first computes the desired COM position based on the virtual support polygon approach. It then uses this along with the robot state, inertia and constant weight parameters to implement a PD controller, called the Balance

Table 5.2: Rewards and Penalties for Training DRL-based Controller

Reward/Penalty	Calculation	Weight
XY-Linear Velocity Tracking	$\exp\left(-\frac{\ \mathbf{v}_{xy}-\mathbf{v}_{xy}^{cmd}\ ^2}{\sigma}\right)$	3
Z-Angular Velocity Tracking	$\exp\left(-\frac{(\omega_z-\omega_z^{cmd})^2}{\sigma}\right)$	1
Feet Air Time	$\sum_{foot} \max(0, t_{air} - 0.1)$	2
Z-Linear Velocity Tracking	$-v_z^2$	-2
XY-Angular Velocity Tracking	$-(\omega_x^2 + \omega_y^2)$	-0.05
Action Rate	$-\ \mathbf{a}_t - \mathbf{a}_{t-1}\ $	-0.01
Joint Torques	$-\sum_i \tau_i^2$	-1e-6
Joint Accelerations	$-\sum_i \ddot{q}_i^2$	-2.5e-7
Undesired Contacts	$-\sum \mathbb{I}(\text{contact})$	-1

Controller by the authors. The dynamics model described in their paper has a simplification of massless legs, which is applicable for robots such as the MIT Cheetah 3, but to THex. Still, to accurately follow the architecture, we have implemented their dynamics model entirely, and used that in the balance controller to calculate the forces needed to support the robot in stance. These forces need to be computed through a convex optimization solver, for which we used OSQP [osqp].

The swing controller makes use of the robot state and other control parameters to calculate the foot trajectory and determine where the feet should be placed. We have used the approach described in [12] to calculate the final foot position and generated a bezier curve trajectory to reach that point. The foothold calculation used by the

authors works well for dog-like robots as it places the foothold on the ground right ahead of the hip frame; however, for robots like the T-Hex, the footholds need to be offset from the hip. We used a constant for this offset. Since the legs are in swing and do not require extensive optimization to find optimal forces, we have used the PD controller described by the authors. The stance and swing forces are then converted to torque commands and sent to the appropriate legs.

5.1.6 Posture Adaptation

Inspired by [15], the essential purpose of this pipeline was that, given raw depth data of the environment, output body configurations with a height Z^* and corresponding leg span S^* which best keep the robot away from collisions in confined spaces.

First, the camera creates a pointcloud of the local environment. This is essentially an RGB image where each pixel has an XYZ coordinate associated to it depending on its location in real life, hence creating a cloud of points. First, these points are projected onto an XY grid and split in half depending on camera height. Each half is run through the grid map algorithm by Fankhauser et al. [26], creating a *floor* layer $F(x, y)$ and a *ceiling* layer $C(x, y)$. Next, for every cell, the available vertical headroom is:

$$h_{avail}(x, y) = C(x, y) - F(x, y) \quad (5.16)$$

Now, cells where the headroom falls below the robot's minimum possible height z_{min} are classified as unsafe for traversal:

$$W(x, y) = \begin{cases} 1 & \text{if } h_{avail}(x, y) < z_{min} \\ 0 & \text{otherwise} \end{cases} \quad (5.17)$$

Now, to obtain the horizontal legroom per cell to the nearest unsafe region, a Euclidean Distance Transform (EDT) is applied to the wall map :

$$w_{avail}(x, y) = \left(\min_{(x_w, y_w) \in W} \sqrt{(x - x_w)^2 + (y - y_w)^2} \right) \times \text{res} \quad (5.18)$$

where res is the map resolution in m/cell.

Now it is appropriate to consider leg span, which we assume is inversely proportional to height and is modeled as:

$$S(Z) = s_{max} - \left(\frac{Z - z_{min}}{z_{max} - z_{min}} \right) (s_{max} - s_{min}) \quad (5.19)$$

where z_{min}, z_{max} are the kinematic height limits and s_{min}, s_{max} are the corresponding span limits.

We then iterate through each candidate height Z in a discrete set $\{Z_1, \dots, Z_{15}\}$, and compute the vertical and horizontal safety margins as follows:

$$M_Z(Z) = h_{avail}(x, y) - Z \quad (5.20)$$

$$M_W(Z) = w_{avail}(x, y) - S(Z) \quad (5.21)$$

which are then scored accordingly:

$$\text{Score}(Z) = \begin{cases} \min(M_Z(Z), M_W(Z)) & \text{if } Z \leq h_{avail} \text{ and } S(Z) \leq w_{avail} \\ -\infty & \text{otherwise} \end{cases} \quad (5.22)$$

The optimal height has the best score of the lot:

$$Z^* = \arg \max_Z \text{Score}(Z) \quad (5.23)$$

and so the corresponding span $S^* = S(Z^*)$ is passed alongside Z^* as posture commands to the locomotion controller.

5.2 Results

The open-loop kinematic gait was successful in walking in flat terrain, with high sensitivity to the choice of gait parameters as well as the gait schedule. However, it was expectedly poor at walking in rough terrain, often getting stuck when going up hill or falling over when going down hill. Testing the open-loop kinematic gait on hardware gave similar results, with the robot successfully walking in flat terrain. A shortcoming of the open-loop kinematic gait is that any disturbance caused by noise or software or hardware latency causes the robot's heading to deviate, which makes the robot gradually turn while walking.

In simulation, the robot was tested with a camera on posture adaptation for multiple cases: a normal region, a low region, a thin region, and a low and thin region. Based on this, it was successfully able to plot out required postures as per the circumstances and then change its posture before entering the region and in the case of the last region refusing to enter; however, it must be kept in mind that this robot was utilizing a kinematic based gait, which was incapable of turning. Hence, while it could adapt its height, it could not make use of traversability for regions best away from the walls, thus hampering its performance.

The DRL-based controller was successful in learning to walk in the random noise rough terrain and random incline terrains up till the highest difficulty, but only

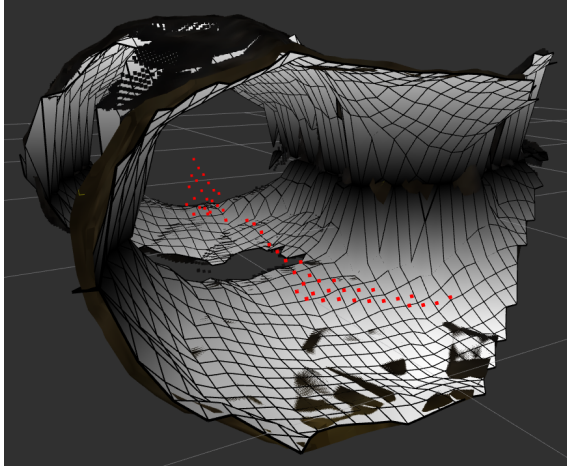


Figure 5.4: A live cave pointcloud, its mapped floors and ceiling, and best z-values per safe zone.

learned to walk in the random blocks rough terrain up till 60% of the highest difficulty. The reason for this is that the random blocks rough terrain can have discontinuities jumps in the terrain’s height, which can be difficult to learn to walk in for a proprioceptive controller that cannot perceive the terrain. When the DRL-based controller was tested in the cave generated by PLUME, it was able to traverse it, but required careful teleoperation to find the right path to take.

The MIT Cheetah 3-like controller did not perform well in even flat terrain. The robot would not be able to lift its legs off the ground to take a step, often shuffling its feet on the ground and not making much forward progress. A suspected reason for this is that the controller uses a simplified dynamic model of the robot that assumes the robot’s legs are massless, which is not the case for T-Hex. Furthermore, our modification to the foothold planning also restricts the sprawl of the robot, which could be chosen dynamically to help the robot balance itself better.



Figure 5.5: Snapshot of the robot walking in the simulated cave.

6. Conclusion and Future Work

We modeled the kinematics of the T-Hex kit and created a simulation package for testing locomotion controllers. We implemented three different controllers, along with a posture adaptation pipeline that gives the nominal base height for locomotion. We integrated the posture adaptation pipeline with the open-loop kinematic gait and tested it in flat terrain and found success in navigating confined spaces. However, the open-loop kinematic gait was expectedly poor at walking in rough terrain. The DRL-based controller was successful in learning to walk in rough terrain, albeit with some limitations in the random blocks rough terrain. The MIT Cheetah 3-like controller did not perform well, likely due to the assumptions made in the dynamic model used for control.

Our next steps include integrating the posture adaptation pipeline with the DRL-based controller, building upon the MIT Cheetah 3-like controller to adapt it for our robot, and testing the DRL-based controller and the MIT Cheetah 3-like controller in hardware. To integrate the posture adaptation pipeline with the DRL-based controller, we plan on training three separate policies for three different nominal base heights, and then use the posture adaptation pipeline to determine which nominal base height to use at any given time. To adapt the MIT Cheetah 3-like controller for our robot, we plan on looking into more accurate dynamic models that take into account the mass of the legs, as well as more robust approaches to foothold selection. Furthermore, as the controller generates torque commands, we also plan to adapt the controller to convert torque commands into position commands [90, 21, 38] so that it can be tested on hardware.

7. Reflection

Learning Reflection: This was another bachelor degree's worth of learning in itself. On the skills side of things, we learned how to practically apply kinematics, reinforcement learning, robot perception, environment simulation, and so many more topics to a real, tangible problem. Aside from that, we had to familiarize ourselves extensively with novel softwares and their quirks particularly ROS2 and all encompassed under it. Of course, it wasn't all hard skills; we learned how to cleanly document our work and regularly present our findings per week in a clear and confident manner.

Team Reflection: Despite having an interdisciplinary team, we organized our efforts based on our strengths. Moreover, we made sure to maintain a clear, shared vision by checking in on each other in times of stress while ensuring we were all agreed on doing what we wanted to do before moving forward. All this proved indispensable in propelling our project forward.

Project Reflection: It is great that our project has fulfilled the individual segments that constitute our deliverables. We had planned on extending perception into the real domain, but due to lack of time as well as poor motor durability, that could not happen. However, on the flip side, there were multiple ambitious parts particularly reinforcement learning we never thought we would be able to accomplish, but we did.

Process Reflection: We were efficient and our task divisions normally clear. To make sure everyone was on the same page, we maintained a OneNote document where each teammate documented whatever work they have done, what work they will do, and any current or potential obstacles. What could have gone better was to concentrate efforts in first accomplishing the general robot pipeline before trying out various approaches for smaller segments.

References

- [1] “(PDF) Probabilistic Terrain Mapping for Mobile Robots With Uncertain Localization”. In: *ResearchGate* (). DOI: 10.1109/LRA.2018.2849506. (Visited on 04/13/2026).
- [2] “(PDF) RHex: A Simple and Highly Mobile Hexapod Robot”. In: *ResearchGate* (). (Visited on 04/13/2026).
- [3] (PDF) *TITAN-XIII: Sprawling-Type Quadruped Robot with Ability of Fast and Energy-Efficient Walking*. https://www.researchgate.net/publication/298909627_TITAN-XIII_sprawling-type-quadruped-robot-with_ability_of_fast_and_energy-efficient_walking. (Visited on 04/13/2026).
- [4] *3D SLAM Using Planar Segments*. <https://www.research-collection.ethz.ch/entities/publication/4f41-4929-9c00-c0ac478dbabf>. (Visited on 04/13/2026).
- [5] *Adaptive Dynamic Walking of a Quadruped Robot on Irregular Terrain Based on Biological Concepts* - Yasuhiro Fukuoka, Hiroshi Kimura, Avis H. Cohen, 2003. https://journals.sagepub.com/doi/abs/10.1177/0278364903022003004?casa_token=3B (Visited on 10/08/2025).
- [6] Philip Arm et al. “SpaceBok: A Dynamic Legged Robot for Space Exploration”. In: *2019 International Conference on Robotics and Automation (ICRA)*. Montreal, QC, Canada: IEEE, May 2019, pp. 6288–6294. ISBN: 978-1-5386-6027-0. DOI: 10.1109/ICRA.2019.8794136. (Visited on 08/24/2025).
- [7] Victor Barasuol et al. “A Reactive Controller Framework for Quadrupedal Locomotion on Challenging Terrain”. In: *2013 IEEE International Conference on Robotics and Automation*. May 2013, pp. 2554–2561. DOI: 10.1109/ICRA.2013.6630926. (Visited on 10/08/2025).
- [8] Dominik Belter, Przemysław Łabęcki, and Piotr Skrzypczyński. “Adaptive Motion Planning for Autonomous Rough Terrain Traversal with a Walking Robot”. In: *Journal of Field Robotics* 33.3 (2016), pp. 337–370. ISSN: 1556-4967. DOI: 10.1002/rob.21610. (Visited on 10/08/2025).
- [9] Marko Bjelonic et al. “Weaver: Hexapod Robot for Autonomous Navigation on Unstructured Terrain”. In: *Journal of Field Robotics* 35.7 (Oct. 2018), pp. 1063–1079. ISSN: 1556-4959, 1556-4967. DOI: 10.1002/rob.21795. (Visited on 04/13/2026).
- [10] Gerardo Bledt. “Policy Regularized Model Predictive Control (PR-MPC) for Robust Legged Locomotion”. In: ().

- [11] Gerardo Blede et al. “Contact Model Fusion for Event-Based Locomotion in Unstructured Terrains”. In: *2018 IEEE International Conference on Robotics and Automation (ICRA)*. Brisbane, QLD: IEEE, May 2018, pp. 4399–4406. ISBN: 978-1-5386-3081-5. DOI: 10.1109/ICRA.2018.8460904. (Visited on 10/19/2025).
- [12] Gerardo Blede et al. “MIT Cheetah 3: Design and Control of a Robust, Dynamic Quadruped Robot”. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Madrid: IEEE, Oct. 2018, pp. 2245–2252. ISBN: 978-1-5386-8094-0. DOI: 10.1109/IROS.2018.8593885. (Visited on 08/30/2025).
- [13] Amanda Bouman et al. “Autonomous Spot: Long-Range Autonomous Exploration of Extreme Environments with Legged Locomotion”. In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2020, pp. 2518–2525. DOI: 10.1109/IROS45743.2020.9341361. (Visited on 11/26/2025).
- [14] Russell Buchanan et al. “Perceptive Whole-Body Planning for Multilegged Robots in Confined Spaces”. In: *Journal of Field Robotics* 38.1 (2021), pp. 68–84. ISSN: 1556-4967. DOI: 10.1002/rob.21974. (Visited on 04/13/2026).
- [15] Russell Buchanan et al. *Walking Posture Adaptation for Legged Robot Navigation in Confined Spaces*. Jan. 2019. DOI: 10.1109/LRA.2019.2899664. arXiv: 1901.10863 [cs]. (Visited on 09/28/2025).
- [16] G. A. Cavagna, N. C. Heglund, and C. R. Taylor. “Mechanical Work in Terrestrial Locomotion: Two Basic Mechanisms for Minimizing Energy Expenditure”. In: *American Journal of Physiology-Regulatory, Integrative and Comparative Physiology* 233.5 (Nov. 1977), R243–R261. ISSN: 0363-6119, 1522-1490. DOI: 10.1152/ajpregu.1977.233.5.R243. (Visited on 10/08/2025).
- [17] Guangrong Chen et al. “Virtual Model Control for Quadruped Robots”. In: *IEEE Access* 8 (2020), pp. 140736–140751. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2020.3013434. (Visited on 10/06/2025).
- [18] Weihai Chen et al. “Locomotion Control with Sensor-Driven Reflex for a Hexapod Robot Walking on Uneven Terrain”. In: *Transactions of the Institute of Measurement and Control* 38.8 (Aug. 2016), pp. 956–970. ISSN: 0142-3312. DOI: 10.1177/0142331215571121. (Visited on 10/08/2025).
- [19] Suyoung Choi et al. “Learning Quadrupedal Locomotion on Deformable Terrain”. In: *Science Robotics* 8.74 (Jan. 2023), eade2256. ISSN: 2470-9476. DOI: 10.1126/scirobotics.ade2256. (Visited on 08/24/2025).

- [20] Petr Čížek, Martin Zoula, and Jan Faigl. “Design, Construction, and Rough-Terrain Locomotion Control of Novel Hexapod Walking Robot With Four Degrees of Freedom Per Leg”. In: *IEEE Access* 9 (2021), pp. 17866–17881. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3053492. (Visited on 10/08/2025).
- [21] Andrea Del Prete et al. “Implementing Torque Control with High-Ratio Gear Boxes and Without Joint-Torque Sensors”. In: *International Journal of Humanoid Robotics* 13.01 (Mar. 2016), p. 1550044. ISSN: 0219-8436, 1793-6942. DOI: 10.1142/S0219843615500449. (Visited on 04/12/2026).
- [22] Jared Di Carlo et al. “Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control”. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2018, pp. 1–9. DOI: 10.1109/IROS.2018.8594448. (Visited on 10/08/2025).
- [23] Yanran Ding, Abhishek Pandala, and Hae-Won Park. “Real-Time Model Predictive Control for Versatile Dynamic Motions in Quadrupedal Robots”. In: *2019 International Conference on Robotics and Automation (ICRA)*. May 2019, pp. 8484–8490. DOI: 10.1109/ICRA.2019.8793669. (Visited on 10/19/2025).
- [24] *DOBOT HEXAPOD - Robot Dog*. <https://funduinoshop.com/en/robotics/dobot-robot-arms/dobot-roboterhunde/dobot-hexapod-robot-dog>. (Visited on 04/13/2026).
- [25] Gen Endo et al. “Learning CPG-based Biped Locomotion with a Policy Gradient Method: Application to a Humanoid Robot”. In: *The International Journal of Robotics Research* 27.2 (Feb. 2008), pp. 213–228. ISSN: 0278-3649. DOI: 10.1177/0278364907084980. (Visited on 10/08/2025).
- [26] P. Fankhauser et al. “Robot-Centric Elevation Mapping with Uncertainty Estimates”. In: *Mobile Service Robotics*. WORLD SCIENTIFIC, July 2014, pp. 433–440. ISBN: 978-981-4623-34-6. DOI: 10.1142/9789814623353_0051. (Visited on 04/13/2026).
- [27] Yasuhiro Fukuoka, Yasushi Habu, and Takahiro Fukui. “A Simple Rule for Quadrupedal Gait Generation Determined by Leg Loading Feedback: A Modeling Study”. In: *Scientific Reports* 5.1 (Feb. 2015), p. 8169. ISSN: 2045-2322. DOI: 10.1038/srep08169. (Visited on 10/08/2025).
- [28] Gabriel Manuel Garcia, Antoine Richard, and Miguel Olivares-Mendez. *PLUME: Procedural Layer Underground Modeling Engine*. Aug. 2025. DOI: 10.48550/arXiv.2508.20926. arXiv: 2508.20926 [cs]. (Visited on 09/26/2025).

- [29] C. Gehring et al. “ANYmal in the Field: Solving Industrial Inspection of an Offshore HVDC Platform with a Quadrupedal Robot”. In: *Field and Service Robotics*. Ed. by Genya Ishigami and Kazuya Yoshida. Vol. 16. Singapore: Springer Singapore, 2021, pp. 247–260. ISBN: 978-981-15-9459-5 978-981-15-9460-1. DOI: 10.1007/978-981-15-9460-1_18. (Visited on 11/26/2025).
- [30] Ruben Grandia et al. “Design and Control of a Bipedal Robotic Character”. In: *Robotics: Science and Systems XX*. July 2024. DOI: 10.15607/RSS.2024.XX.103. arXiv: 2501.05204 [cs]. (Visited on 11/26/2025).
- [31] Andrei Herdt, Nicolas Perrin, and Pierre-Brice Wieber. “Walking without Thinking about It”. In: *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Oct. 2010, pp. 190–195. DOI: 10.1109/IRoS.2010.5654429. (Visited on 10/08/2025).
- [32] David Hoeller et al. *ANYmal Parkour: Learning Agile Navigation for Quadrupedal Robots*. June 2023. DOI: 10.48550/arXiv.2306.14874. arXiv: 2306.14874 [cs]. (Visited on 08/24/2025).
- [33] Armin Hornung et al. “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees”. In: *Autonomous Robots* 34.3 (Apr. 2013), pp. 189–206. ISSN: 0929-5593, 1573-7527. DOI: 10.1007/s10514-012-9321-0. (Visited on 04/13/2026).
- [34] Nan Hu et al. “Constrained Model Predictive Control for a Hexapod Robot Walking on Irregular Terrain”. In: *Journal of Intelligent & Robotic Systems* 94.1 (Apr. 2019), pp. 179–201. ISSN: 1573-0409. DOI: 10.1007/s10846-018-0827-3. (Visited on 10/08/2025).
- [35] Marco Hutter et al. “ANYmal - a Highly Mobile and Dynamic Quadrupedal Robot”. In: *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Daejeon, South Korea: IEEE, Oct. 2016, pp. 38–44. ISBN: 978-1-5090-3762-9. DOI: 10.1109/IRoS.2016.7758092. (Visited on 08/30/2025).
- [36] Marco Hutter et al. “STARLETH: A COMPLIANT QUADRUPEDAL ROBOT FOR FAST, EFFICIENT, AND VERSATILE LOCOMOTION”. In: *Adaptive Mobile Robotics*. WORLD SCIENTIFIC, Sept. 2012, pp. 483–490. ISBN: 978-981-4415-94-1 978-981-4415-95-8. DOI: 10.1142/9789814415958_0062. (Visited on 11/26/2025).
- [37] *Introduction to Autonomous Mobile Robots*. <https://mitpress.mit.edu/9780262015356/introduction-to-autonomous-mobile-robots/>. (Visited on 04/06/2026).

- [38] Oussama Khatib et al. “Torque-Position Transformer for Task Control of Position Controlled Robots”. In: *2008 IEEE International Conference on Robotics and Automation*. Pasadena, CA, USA: IEEE, May 2008, pp. 1729–1734. ISBN: 978-1-4244-1646-2. DOI: 10.1109/ROBOT.2008.4543450. (Visited on 04/12/2026).
- [39] Joonyoung Kim et al. “PADWQ - Design and Control of a Open-Source, Low Cost, 3D Printed Dynamic Quadruped Robot”. In: *Applied Sciences* 11.9 (Apr. 2021), p. 3762. ISSN: 2076-3417. DOI: 10.3390/app11093762. (Visited on 08/24/2025).
- [40] Taesoo Kim et al. “Deep Reinforcement Learning Control of a Hexapod Robot”. In: *Actuators* 15.1 (Jan. 2026). ISSN: 2076-0825. DOI: 10.3390/act15010033. (Visited on 03/25/2026).
- [41] Edin Koco, Damir Mirkovic, and Zdenko Kovačić. “Hybrid Compliance Control for Locomotion of Electrically Actuated Quadruped Robot”. In: *Journal of Intelligent & Robotic Systems* 94.3 (June 2019), pp. 537–563. ISSN: 1573-0409. DOI: 10.1007/s10846-018-0777-9. (Visited on 10/15/2025).
- [42] Hendrik Kolvenbach et al. *Traversing Steep and Granular Martian Analog Slopes With a Dynamic Quadrupedal Robot*. June 2021. DOI: 10.48550/arXiv.2106.01974. arXiv: 2106.01974 [cs]. (Visited on 09/27/2025).
- [43] Joonho Lee et al. “Learning Quadrupedal Locomotion over Challenging Terrain”. In: *Science Robotics* 5.47 (Oct. 2020), eabc5986. ISSN: 2470-9476. DOI: 10.1126/scirobotics.abc5986. arXiv: 2010.11251 [cs]. (Visited on 08/24/2025).
- [44] Wenlu Li et al. “Parameter Tuning of CPGs for Hexapod Gaits Based on Genetic Algorithm”. In: *2015 IEEE 10th Conference on Industrial Electronics and Applications (ICIEA)*. June 2015, pp. 45–50. DOI: 10.1109/ICIEA.2015.7334082. (Visited on 09/24/2025).
- [45] Yubin Liu et al. “Research on the Posture Control Method of Hexapod Robot for Rugged Terrain”. In: *Applied Sciences* 10.19 (Jan. 2020), p. 6725. ISSN: 2076-3417. DOI: 10.3390/app10196725. (Visited on 10/08/2025).
- [46] Gabriel B. Margolis et al. *Rapid Locomotion via Reinforcement Learning*. May 2022. DOI: 10.48550/arXiv.2205.02824. arXiv: 2205.02824 [cs]. (Visited on 02/02/2026).
- [47] Takahiro Miki et al. “Learning Robust Perceptive Locomotion for Quadrupedal Robots in the Wild”. In: *Science Robotics* 7.62 (Jan. 2022), eabk2822. ISSN: 2470-9476. DOI: 10.1126/scirobotics.abk2822. arXiv: 2201.08117 [cs]. (Visited on 08/17/2025).

- [48] Takahiro Miki et al. *Learning to Walk in Confined Spaces Using 3D Representation*. Feb. 2024. DOI: 10.48550/arXiv.2403.00187. arXiv: 2403.00187 [cs]. (Visited on 04/13/2026).
- [49] *mit-biomimetics/Cheetah-Software*. original-date: 2019-06-17T20:27:11Z. Apr. 2026. URL: <https://github.com/mit-biomimetics/Cheetah-Software> (visited on 04/14/2026).
- [50] I. Made Aswin Nahrendra, Byeongho Yu, and Hyun Myung. *DreamWaQ: Learning Robust Quadrupedal Locomotion With Implicit Terrain Imagination via Deep Reinforcement Learning*. Mar. 2023. DOI: 10.48550/arXiv.2301.10602. arXiv: 2301.10602 [cs]. (Visited on 08/26/2025).
- [51] Komeil Nosrati et al. *Regularized Model Predictive Control*. May 2025. DOI: 10.48550/arXiv.2505.12977. arXiv: 2505.12977 [eess]. (Visited on 10/14/2025).
- [52] *Novo Nordisk Explores Spot’s Role in Pharmaceutical Production*. (Visited on 11/26/2025).
- [53] NVIDIA et al. *Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning*. Nov. 2025. DOI: 10.48550/arXiv.2511.04831. arXiv: 2511.04831 [cs]. (Visited on 11/26/2025).
- [54] Helen Oleynikova et al. “Voxblox: Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning”. In: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Sept. 2017, pp. 1366–1373. DOI: 10.1109/IROS.2017.8202315. arXiv: 1611.03631 [cs]. (Visited on 04/13/2026).
- [55] Alex Oshin, Hassan Almubarak, and Evangelos Theodorou. “Differentiable Robust Model Predictive Control”. In: *Robotics: Science and Systems XX*. Robotics: Science and Systems Foundation, July 2024. ISBN: 979-8-9902848-0-7. DOI: 10.15607/RSS.2024.XX.003. (Visited on 10/08/2025).
- [56] Dai Owaki and Akio Ishiguro. “A Quadruped Robot Exhibiting Spontaneous Gait Transitions from Walking to Trotting to Galloping”. In: *Scientific Reports* 7.1 (Mar. 2017), p. 277. ISSN: 2045-2322. DOI: 10.1038/s41598-017-00348-9. (Visited on 10/08/2025).

- [57] Chang-Soo Park and Jong-Hwan Kim. “Stable Modifiable Walking Pattern Algorithm with Constrained Optimized Central Pattern Generator”. In: *Robot Intelligence Technology and Applications 2012: An Edition of the Presented Papers from the 1st International Conference on Robot Intelligence Technology and Applications*. Ed. by Jong-Hwan Kim et al. Berlin, Heidelberg: Springer, 2013, pp. 223–230. ISBN: 978-3-642-37374-9. DOI: 10.1007/978-3-642-37374-9_22. (Visited on 10/08/2025).
- [58] Hae-Won Park, Patrick M. Wensing, and Sangbae Kim. “Online Planning for Autonomous Running Jumps Over Obstacles in High-Speed Quadrupeds”. In: *Park* (July 2015). ISSN: 2330-7668. (Visited on 10/08/2025).
- [59] Ioannis Poulakakis, Evangelos Papadopoulos, and Martin Buehler. “On the Stability of the Passive Dynamics of Quadrupedal Running with a Bounding Gait”. In: *The International Journal of Robotics Research* 25.7 (July 2006), pp. 669–687. ISSN: 0278-3649, 1741-3176. DOI: 10.1177/0278364906066768. (Visited on 11/26/2025).
- [60] Jerry E. Pratt. “Virtual Model Control of a Biped Walking Robot”. In: (Dec. 1995). (Visited on 10/08/2025).
- [61] Marc Raibert et al. “BigDog, the Rough-Terrain Quadruped Robot”. In: *IFAC Proceedings Volumes* 41.2 (2008), pp. 10822–10825. ISSN: 14746670. DOI: 10.3182/20080706-5-KR-1001.01833. (Visited on 11/26/2025).
- [62] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, 1986. ISBN: 978-0-262-18117-4.
- [63] Marc H. Raibert. “Trotting, Pacing and Bounding by a Quadruped Robot”. In: *Journal of Biomechanics*. International Society of Biomechanics 23 (Jan. 1990), pp. 79–98. ISSN: 0021-9290. DOI: 10.1016/0021-9290(90)90043-3. (Visited on 10/08/2025).
- [64] Guanjiao Ren et al. “Multiple Chaotic Central Pattern Generators with Learning for Legged Locomotion and Malfunction Compensation”. In: *Information Sciences* 294 (Feb. 2015), pp. 666–682. ISSN: 00200255. DOI: 10.1016/j.ins.2014.05.001. arXiv: 1407.3269 [cs]. (Visited on 09/26/2025).
- [65] Nikita Rudin et al. *Advanced Skills by Learning Locomotion and Local Navigation End-to-End*. Sept. 2022. DOI: 10.48550/arXiv.2209.12827. arXiv: 2209.12827 [cs]. (Visited on 11/26/2025).

- [66] Nikita Rudin et al. “Cat-like Jumping and Landing of Legged Robots in Low-gravity Using Deep Reinforcement Learning”. In: *IEEE Transactions on Robotics* 38.1 (Feb. 2022), pp. 317–328. ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TR0.2021.3084374. arXiv: 2106.09357 [cs]. (Visited on 08/15/2025).
- [67] Nikita Rudin et al. “Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning”. In: *Proceedings of the 5th Conference on Robot Learning*. PMLR, Jan. 2022, pp. 91–100. (Visited on 01/29/2026).
- [68] *Running on Four Legs as They Are One*. https://www.ri.cmu.edu/pub_files/pub3/raibert_m_h_19 (Visited on 10/15/2025).
- [69] John Schulman et al. *Proximal Policy Optimization Algorithms*. Aug. 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs]. (Visited on 02/06/2026).
- [70] Claudio Semini et al. “HyQ - A Dynamic Locomotion Research Platform”. In: ().
- [71] Shuo. *ShuoYangRobotics/A1-QP-MPC-Controller*. original-date: 2022-03-21T02:09:03Z. Apr. 2026. URL: <https://github.com/ShuoYangRobotics/A1-QP-MPC-Controller> (visited on 04/14/2026).
- [72] *Spot*. (Visited on 10/04/2025).
- [73] *Spot Robot Assists Police and First Responder Operations as Part of the Tactical Bubble | FlyMotion*. <https://www.flymotionus.com/posts/spot-robot-assists-police-and-first-responder-operations-as-part-of-the-tactical-bubble>. (Visited on 11/26/2025).
- [74] *Spot to the Rescue*. (Visited on 11/26/2025).
- [75] Jianbo Sun et al. “Hexapod Robot Kinematics Modeling and Tripod Gait Design Based on the Foot End Trajectory”. In: *2017 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. Macau: IEEE, Dec. 2017, pp. 2611–2616. ISBN: 978-1-5386-3742-5. DOI: 10.1109/ROBIO.2017.8324813. (Visited on 11/26/2025).
- [76] Yinghan Sun. *yinghansun/pympc-quadruped*. original-date: 2023-04-25T07:02:34Z. Apr. 2026. URL: <https://github.com/yinghansun/pympc-quadruped> (visited on 04/14/2026).
- [77] *Sw_urdf_exporter - ROS Wiki*. https://wiki.ros.org/sw_urdf_exporter. (Visited on 04/12/2026).
- [78] *T-Hex 3DoF Hexapod*. <https://wiki.lynxmotion.com/info/wiki/lynxmotion/view/ses-v1/ses-v1-robots/ses-v1-3-4-dof-hexapods/t-hex-3/>. (Visited on 09/27/2025).

- [79] M. Taha, A. Badawy, and M. Hegazy. “Developing Hexapod Locomotion Through Reinforcement Learning”. In: *2025 Intelligent Methods, Systems, and Applications (IMSA)*. July 2025, pp. 61–66. DOI: 10.1109/IMSA65733.2025.11166981. (Visited on 03/25/2026).
- [80] Jie Tan et al. *Sim-to-Real: Learning Agile Locomotion For Quadruped Robots*. May 2018. DOI: 10.48550/arXiv.1804.10332. arXiv: 1804.10332 [cs]. (Visited on 10/08/2025).
- [81] Ann L Torres. “Virtual Model Control of a Hexapod Walking Robot”. In: ().
- [82] Marco Tranzatto et al. *CERBERUS: Autonomous Legged and Aerial Robotic Exploration in the Tunnel and Urban Circuits of the DARPA Subterranean Challenge*. Jan. 2022. DOI: 10.48550/arXiv.2201.07067. arXiv: 2201.07067 [cs]. (Visited on 08/24/2025).
- [83] *Unitree B2*. <https://shop.unitree.com/products/unitree-b2>. (Visited on 04/13/2026).
- [84] *Unitree Firefighting Robots Transform Fire Rescue Operations | Digital Watch Observatory*. Nov. 2025. (Visited on 11/26/2025).
- [85] *Unitree Go2*. <https://shop.unitree.com/products/unitree-go2>. (Visited on 04/13/2026).
- [86] *Vision 60 | Ghost Robotics*. <https://www.ghostrobotics.io/vision-60>. (Visited on 04/13/2026).
- [87] Pierre-brice Wieber. “Trajectory Free Linear Model Predictive Control for Stable Walking in the Presence of Strong Perturbations”. In: *2006 6th IEEE-RAS International Conference on Humanoid Robots*. Dec. 2006, pp. 137–142. DOI: 10.1109/ICHR.2006.321375. (Visited on 10/08/2025).
- [88] Huixiang Xie et al. “An Intuitive Approach for Quadruped Robot Trotting Based on Virtual Model Control”. In: *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering* 229.4 (Apr. 2015), pp. 342–355. ISSN: 0959-6518. DOI: 10.1177/0959651814562620. (Visited on 10/08/2025).
- [89] *Yerbert/DingoQuadruped: Base Code for the Dingo Quadruped; Modified from Stanford Pupper and Notspot Repositories. Includes Integration with ROS Noetic and a Simulation of the Dingo*. <https://github.com/Yerbert/DingoQuadruped>. (Visited on 08/24/2025).

- [90] Gabriela Zepeda, Rafael Kelly, and Carmen Monroy. “A Theoretical Conversion of Torque Controllers for Handling a Position Servo-actuated Robot Manipulator Model”. In: *International Journal of Control, Automation and Systems* 22.4 (Apr. 2024), pp. 1301–1312. ISSN: 2005-4092. DOI: 10.1007/s12555-022-1243-9. (Visited on 04/12/2026).
- [91] Jiaqi Zhang et al. “Trot Gait Design and CPG Method for a Quadruped Robot”. In: *Journal of Bionic Engineering* 11.1 (Jan. 2014), pp. 18–25. ISSN: 1672-6529. DOI: 10.1016/S1672-6529(14)60016-0. (Visited on 10/08/2025).